PEMROGRAMAN BERORIENTASI OBJEK 2026

Koleksi dan *Generics*

Memilih wadah yang tepat untuk banyak objek, lalu memberinya label tipe agar tidak salah isi

PERTEMUAN

12

dari 16

Program Studi Informatika · Universitas Mulawarman
Jalur praktik: Python dan Java

Tujuan Pembelajaran

Setelah pertemuan ini, kamu mampu:

1. Menjelaskan ciri empat koleksi utama (*list*, *tuple*, set, dan map/dict) lalu memilih yang tepat untuk sebuah kebutuhan.
2. Membedakan *interface* koleksi dengan wujudnya, dan menjelaskan mengapa urutan isi `HashSet`/`HashMap` tidak boleh diandalkan.
3. Mengurutkan objek dengan **urutan alami** maupun **urutan khusus**, termasuk memakai *lambda*.
4. Menjelaskan aturan kesamaan objek dan alasan `__eq__`/`equals` harus berpasangan dengan `__hash__`/`hashCode`.
5. Membuat dan memakai kelas generik `Pasangan[A, B]` / `Pasangan<A, B>`, serta membandingkan pemeriksaan tipe di Python dan Java.

Mengingat Kembali



Dari pertemuan sebelumnya

- Pertemuan 05: *list* berisi objek, yaitu `list` di Python dan `ArrayList` di Java. `List<Menu>` dibaca "*list* berisi `Menu`".
- Pertemuan 07: secara bawaan, dua objek dianggap sama **hanya jika** keduanya objek yang sama persis.
- Pertemuan 10: *interface* adalah **kontrak**. Di Java, `List`, `Set`, dan `Map` ternyata juga *interface*.
- Pertemuan 11: `cari_menu()`/`cariMenu()` memeriksa menu **satu per satu** dari depan sebelum melempar *exception*.

Agenda Hari Ini

1. Empat wadah dan kontraknya
2. Urutan isi dan memilih wadah
3. Mengurutkan objek dan *lambda*
4. Kesamaan objek dan *generics*
5. Studi kasus KedaiKita

BAGIAN 1

1

Empat Wadah dan Kontraknya

List, *tuple*, set, dan map: setiap wadah punya kepribadian sendiri.

Kasir yang Kewalahan

Pertanyaan	Kebutuhan sebenarnya	Wadah yang cocok
Harga M04 berapa?	mencari lewat **kunci** (kode)	map / dict
Kode M01 dipakai dua kali	menolak kunci **kembar**	map / dict, set
Kategori apa saja?	data **unik** tanpa kembar	set
Urutkan dari termurah	**mengurutkan** objek	*list* + aturan urutan
Menu paling laris?	**menghitung** per menu, lalu mengurutkan	map / dict + *list*

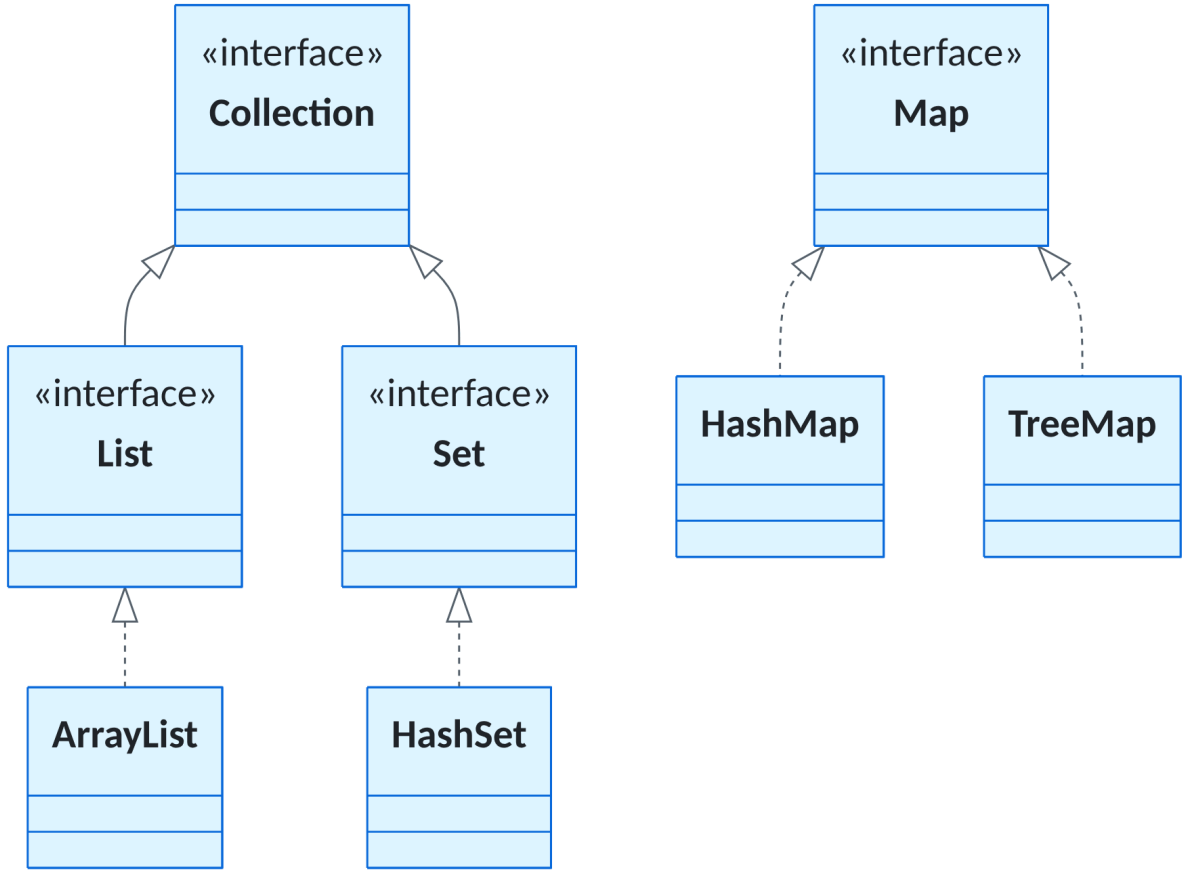
Koleksi (*collection*): objek yang tugasnya menyimpan dan mengelola sekumpulan objek lain.

Empat Wadah Utama

Wadah	Analogi	Ciri utama
List	daftar belanja atau antrean bernomor	berurutan, boleh kembar, bisa diubah, diakses lewat indeks
Tuple (Java: *list* tetap)	data yang sudah tercetak di KTP	berurutan, tetapi isinya **tidak bisa diubah** (*immutable*)
Set	daftar peserta unik sebuah lokakarya	**tanpa data kembar**, cepat menjawab "ada atau tidak?", urutan tidak dijamin
Map (Python: dict)	buku telepon atau loker berkunci	pasangan **kunci → nilai**, kunci tidak boleh kembar

Java tidak punya *tuple*. Padanan terdekatnya adalah *list* tetap buatan `List.of(...)`.

Kontrak dan Wujud Koleksi Java



Tulis tipe kontrak di kiri dan wujud di kanan: `List<Menu> daftar = new ArrayList<>();`.

Set: Kumpulan tanpa Data Kembar

`pendaftar` berisi enam formulir; Ayu dan Bima mengisi dua kali. Set membuang data kembar secara otomatis.

Python python/contoh/contoh02_set_himpunan.py

```python
peserta_python: set[str] = set(pendaftar)
peserta_java = {"Citra", "Eka", "Ayu", "Fajar"}
# ... (dilewati)
peserta_python.add("Eka")
peserta_python.add("Ayu")
# ... (dilewati)
cetak("Ikut keduanya", peserta_python & peserta_java)
cetak("Semua peserta", peserta_python | peserta_java)
cetak("Hanya Python", peserta_python - peserta_java)
if "Fajar" not in peserta_python:
    print("Fajar belum ikut workshop Python")
```

Java java/contoh/Contoh02SetHimpunan.java

```java
Set<String> pesertaPython = new HashSet<>(pendaftar);
// ... (dilewati)
pesertaPython.add("Eka");
pesertaPython.add("Ayu");
// ... (dilewati)
Set<String> irisan = new HashSet<>(pesertaPython);
irisan.retainAll(pesertaJava);
// ... (dilewati)
Set<String> gabungan = new HashSet<>(pesertaPython);
gabungan.addAll(pesertaJava);
// ... (dilewati)
if (!pesertaPython.contains("Fajar")) {
    System.out.println("Fajar belum ikut workshop Python");
}
```

BAGIAN 2

2

Urutan Isi dan Memilih Wadah

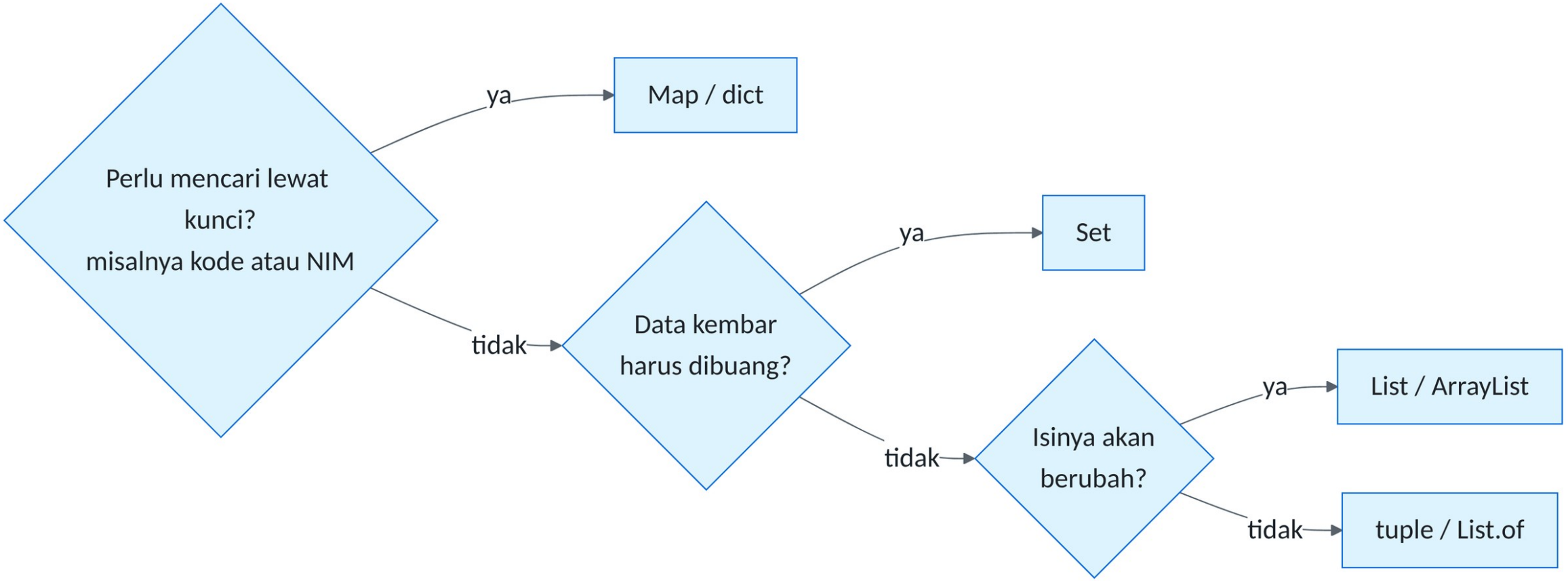
Jangan mengandalkan kebetulan, lalu tiga pertanyaan untuk memilih wadah.

Urutan Isi: Jangan Mengandalkan Kebetulan

Wadah	Urutan saat isinya dijelajahi
Python `list`/`tuple`, Java `ArrayList`/`List.of`	sesuai indeks (urutan dimasukkan)
Python `dict`	sesuai urutan kunci **pertama kali dimasukkan** (dijamin sejak Python 3.7)
Python `set`	**tidak dijamin**
Java `HashSet`/`HashMap`/`Set.of`/`Map.of`	**tidak dijamin**
Java `LinkedHashSet`/`LinkedHashMap`	sesuai urutan dimasukkan
Java `TreeSet`/`TreeMap`	selalu **urut** (abjad, angka, atau aturan yang kita berikan)

Jika urutan penting, pilih wadah yang menjamin urutan, atau **urutkan dulu** sebelum ditampilkan.

Memilih Koleksi yang Tepat



Setelah jenisnya terpilih, tentukan kebutuhan urutannya.

BAGIAN 3

3

Mengurutkan Objek

Komputer tidak bisa menebak cara mengurutkan objek; kita harus memberi tahu aturannya.

Urutan Alami dan Urutan Khusus

Aspek	Urutan alami	Urutan khusus
Jumlahnya per kelas	satu	sebanyak yang dibutuhkan
Ditulis di	dalam kelas itu sendiri	di luar kelas, saat mengurutkan
Python	*method* `__lt__` (“kurang dari”)	argumen `key=` berisi fungsi atau *lambda*
Java	`implements Comparable<T>` + `compareTo()`	objek `Comparator` (*lambda*, `Comparator.comparing`)
Contoh di KedaiKita	menu: harga termurah dulu	menu: nama A—Z

Di Java, `a.compareTo(b)` mengembalikan negatif (`a` sebelum `b`), nol (setara), atau positif (`a` sesudah `b`).

Lambda: Fungsi Kecil tanpa Nama

Lambda

Fungsi kecil **tanpa nama** yang ditulis langsung di tempat ia dibutuhkan.

- Python: `lambda m: m.harga`. Java: `m -> m.getHarga()`. Keduanya berarti "diberi menu `m`, kembalikan harganya".
- *Method reference* Java, misalnya `Menu::getHarga`, adalah singkatan *lambda* yang hanya memanggil satu *method*.
- `sorted(daftar, key=...)` membuat *list* baru, sedangkan `daftar.sort(...)` mengurutkan *list* itu sendiri.

Urutan Alami dan Urutan Khusus: Lomba Lari

Urutan alami pelari: tercepat dulu; jika waktunya sama, nomor dada kecil dulu. Absensi memakai urutan khusus: nama A—Z.

Python python/contoh/contoh06_urutan_alami.py

```python
def __lt__(self, lain: "Pelari") -> bool:
    if self._waktu_detik != lain._waktu_detik:
        return self._waktu_detik < lain._waktu_detik
    return self._nomor < lain._nomor
# ... (dilewati)
for nomor, pelari in enumerate(sorted(hasil), start=1):
    print(f"{nomor}. {pelari}")
# ... (dilewati)
absensi = sorted(hasil, key=lambda p: p.nama)
```

Java java/contoh/Contoh06Comparable.java

```java
public int compareTo(Pelari lain) {
    if (waktuDetik != lain.waktuDetik) {
        return Integer.compare(waktuDetik, lain.waktuDetik);
    }
    return Integer.compare(nomor, lain.nomor);
}
// ... (dilewati)
Collections.sort(peringkat);
for (int i = 0; i < peringkat.size(); i++) {
    System.out.println((i + 1) + ". " + peringkat.get(i));
}
// ... (dilewati)
absensi.sort(Comparator.comparing(Pelari::getNama));
```

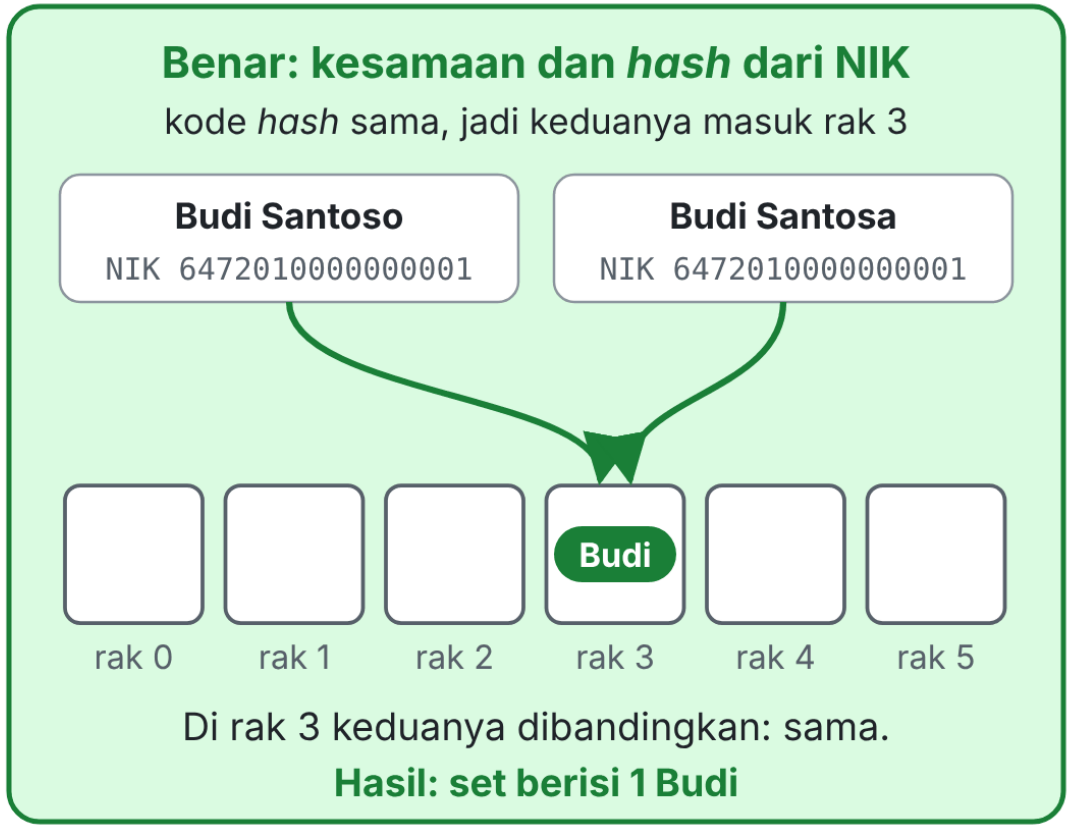
BAGIAN 4

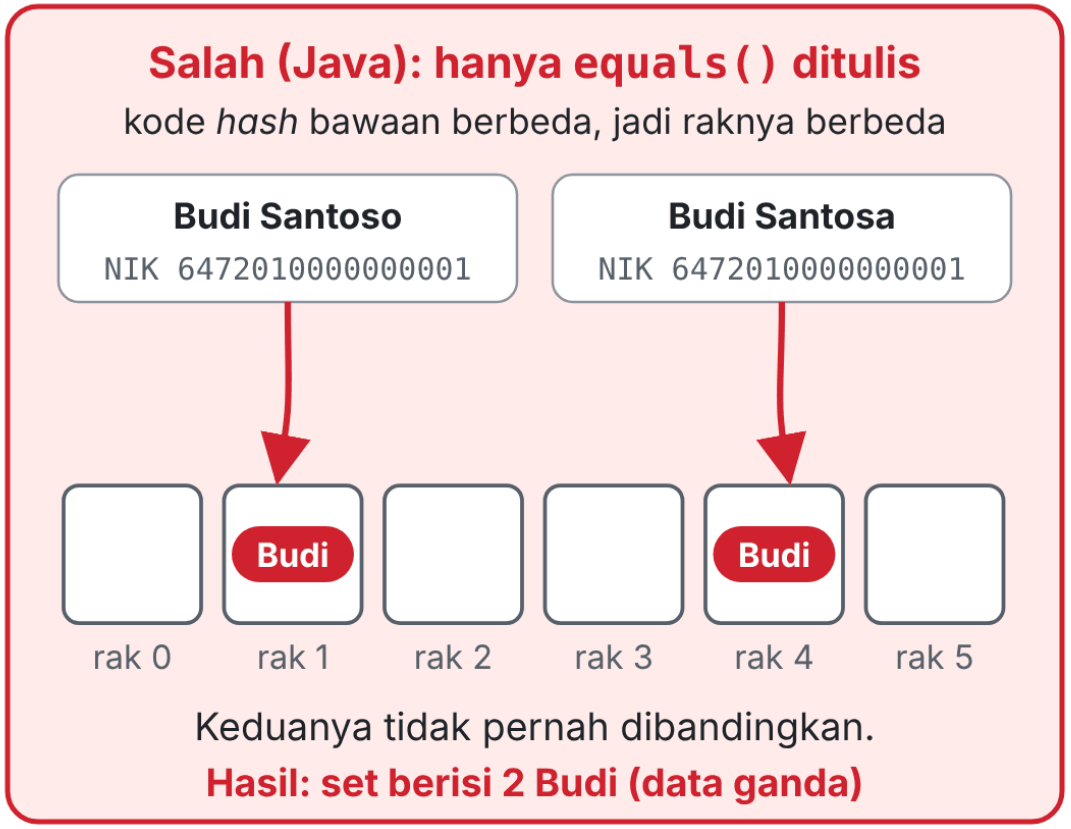
4

Kesamaan Objek dan *Generics*

Kapan dua objek dianggap sama, dan wadah berlabel agar tidak salah isi.

Mengapa Kesamaan Harus Berpasangan dengan *Hash*?





Dua objek yang dianggap sama **harus** punya kode *hash* yang sama. Tulis `__eq__` bersama `__hash__`, atau `equals()` bersama `hashCode()`.

Kesamaan Objek: `__eq__`/`__hash__` dan `equals()`/`hashCode()`

Dua pemilih dianggap sama jika NIK-nya sama, dan kode *hash*-nya juga dihitung dari NIK.

Python python/contoh/contoh07_kesamaan_objek.py

```python
def __eq__(self, lain) -> bool:
    if not isinstance(lain, Pemilih):
        return NotImplemented
    return self._nik == lain._nik

def __hash__(self) -> int:
    return hash(self._nik)
# ... (dilewati)
b1 = Pemilih("6472010000000001", "Budi Santoso")
b2 = Pemilih("6472010000000001", "Budi Santosa")
# ... (dilewati)
dpt: set[Pemilih] = set()
dpt.add(b1)
dpt.add(b2)
```

Java java/contoh/Contoh07EqualsHashCode.java

```java
@Override
public boolean equals(Object lain) {
    if (!(lain instanceof Pemilih)) {
        return false;
    }
    Pemilih pemilihLain = (Pemilih) lain;
    return nik.equals(pemilihLain.nik);
}

@Override
public int hashCode() {
    return nik.hashCode();
}
```

Generics: Wadah Berlabel

Generics

Cara menulis kelas atau fungsi dengan **parameter tipe** (biasanya ditulis T) sehingga tipe isinya baru ditentukan saat dipakai, misalnya `list[Menu]` atau `List<Menu>`.

- Python: label tipe hanya **petunjuk** (*type hint*) dan tidak diperiksa saat program berjalan.
- Java: label tipe adalah **aturan**; isi yang salah tipe ditolak *compiler* sebelum program berjalan.
- Khusus Java: angka disimpan sebagai `Integer`, bukan `int`, dan dibungkus otomatis (*autoboxing*).

Kelas Generik `Pasangan[A, B]` dan `Pasangan<A, B>`

Satu kelas, banyak kegunaan: (kode menu, jumlah porsi) atau (nama menu, porsi terjual).

Python python/studi-kasus/pasangan.py

```python
from typing import Generic, TypeVar

A = TypeVar("A")
B = TypeVar("B")

class Pasangan(Generic[A, B]):
    # ... (dilewati)
    def __init__(self, pertama: A, kedua: B):
        self._pertama = pertama
        self._kedua = kedua

    @property
    def pertama(self) -> A:
        return self._pertama
```

Java java/studi-kasus/Pasangan.java

```java
public class Pasangan<A, B> {
    private A pertama;
    private B kedua;

    public Pasangan(A pertama, B kedua) {
        this.pertama = pertama;
        this.kedua = kedua;
    }

    public A getPertama() {
        return pertama;
    }
    // ... (dilewati)
}
```

STUDI KASUS

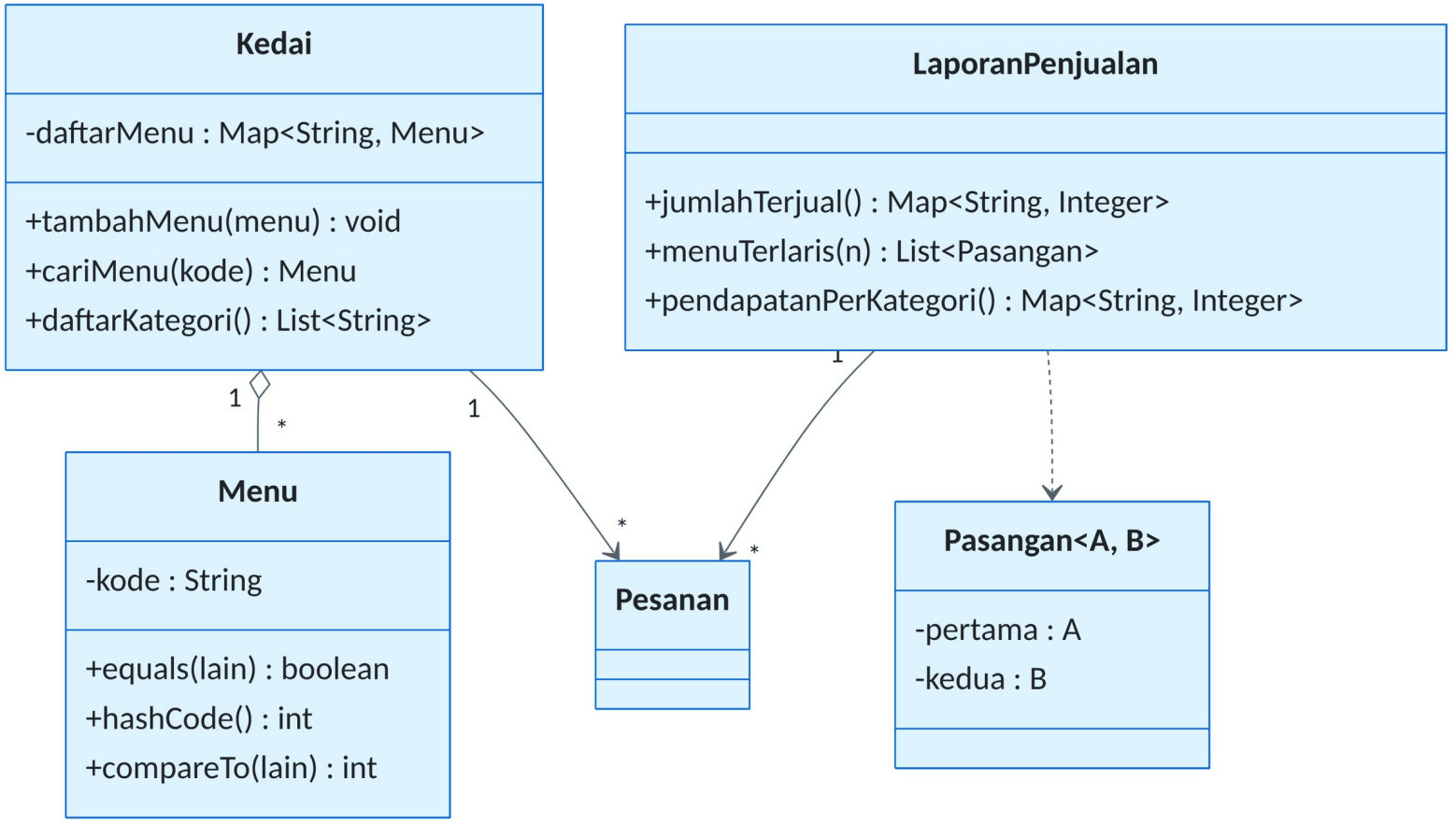


Studi Kasus KedaiKita

Menjawab pertanyaan-pertanyaan Mbak Sari, kasir KedaiKita, dengan koleksi yang tepat.

Rancangan KedaiKita Pertemuan 12

Studi kasus KedaiKita



Nama memakai gaya Java. Di Python: `daftar_menu`, `cari_menu`, serta `__eq__`/`__hash__`/`__lt__`.

`Kedai` Menyimpan Menu di Map

Studi kasus KedaiKita

Kode menu menjadi kunci: pencarian langsung lewat kunci, dan kode ganda ditolak.

Python python/studi-kasus/kedai.py

```python
class Kedai(DapatDicetak):
    def __init__(self, nama: str):
        # ... (dilewati)
        self._daftar_menu: dict[str, Menu] = {}
    # ... (dilewati)
    def tambah_menu(self, menu: Menu):
        if menu.kode in self._daftar_menu:
            raise ValueError(f"Kode {menu.kode} sudah dipakai")
        self._daftar_menu[menu.kode] = menu

    def cari_menu(self, kode: str) -> Menu:
        # ... (dilewati)
        if kode not in self._daftar_menu:
            raise MenuTidakDitemukanError(kode)
        return self._daftar_menu[kode]
```

Java java/studi-kasus/Kedai.java

```java
public class Kedai implements DapatDicetak {
    private Map<String, Menu> daftarMenu;
    // ... (dilewati)
    public void tambahMenu(Menu menu) {
        if (daftarMenu.containsKey(menu.getKode())) {
            // ... lempar IllegalArgumentException
        }
        daftarMenu.put(menu.getKode(), menu);
    }
    public Menu cariMenu(String kode) throws ... {
        if (!daftarMenu.containsKey(kode)) {
            throw new MenuTidakDitemukanException(kode);
        }
        return daftarMenu.get(kode);
    }
}
```

Menu: Kode *Hash* dan Urutan Alami

Studi kasus KedaiKita

Dua menu dianggap sama jika **kodenya** sama, jadi *hash* dihitung dari kode. Urutan alaminya: harga termurah dulu.

Python python/studi-kasus/menu.py

```python
class Menu:
    # ... (dilewati)
    def __hash__(self) -> int:
        return hash(self._kode)

    def __lt__(self, lain: "Menu") -> bool:
        # ... (dilewati)
        if self._harga != lain._harga:
            return self._harga < lain._harga
        return self._kode < lain._kode
```

Java java/studi-kasus/Menu.java

```java
public class Menu implements Comparable<Menu> {
    // ... (dilewati)
    @Override
    public int hashCode() {
        return kode.hashCode();
    }
    // ... (dilewati)
    @Override
    public int compareTo(Menu lain) {
        if (harga != lain.harga) {
            return Integer.compare(harga, lain.harga);
        }
        return kode.compareTo(lain.kode);
    }
}
```

Miskonsepsi yang Sering Terjadi

Anggapan keliru	Yang benar
“Semua koleksi menyimpan isi sesuai urutan saya memasukkannya.”	`list` dan `dict` Python menjaga urutan masuk, tetapi `set` tidak. `HashSet` dan `HashMap` Java tidak menjamin urutan; pakai `LinkedHashMap`/`TreeMap` atau urutkan dulu.
“Cukup menulis `__eq__`/`equals()` agar set membuang data kembar.”	Belum cukup. Tanpa `__hash__`/`hashCode()` dari data yang sama, Python menolak objeknya masuk set, sedangkan Java diam-diam membiarkan data kembar.
“*Type hint* `list[int]` membuat Python menolak teks.”	Python tidak memeriksa *type hint* saat program berjalan; yang membacanya manusia dan alat bantu seperti VS Code atau mypy. *Compiler* Java benar-benar menolaknya.

Kuis Cepat

Panitia seminar mencatat NIM peserta yang memindai QR di pintu masuk. Yang memindai dua kali tetap dihitung **sekali**, dan panitia sering bertanya “apakah NIM ini sudah tercatat?”. Koleksi yang paling tepat adalah …

A. *list* karena bisa menampung data kembar

B. set karena otomatis membuang data kembar dan cepat menjawab “ada atau tidak”

C. *tuple* karena isinya tidak bisa diubah

D. map dengan kunci nomor urut pindai

Kuis Cepat: Jawaban

Panitia seminar mencatat NIM peserta yang memindai QR di pintu masuk. Yang memindai dua kali tetap dihitung **sekali**, dan panitia sering bertanya “apakah NIM ini sudah tercatat?”. Koleksi yang paling tepat adalah …

A. *list* karena bisa menampung data kembar

✓ **B. set karena otomatis membuang data kembar dan cepat menjawab “ada atau tidak”**

C. *tuple* karena isinya tidak bisa diubah

D. map dengan kunci nomor urut pindai

Jawaban: B

Set menolak data kembar dan memeriksa keanggotaan dengan cepat lewat kode *hash*. *List* (A) menyimpan hasil pindai ganda sehingga peserta dihitung dua kali, dan pemeriksaannya satu per satu. *Tuple* (C) tidak bisa ditambah saat peserta baru datang. Map dengan kunci nomor urut pindai (D) tetap menyimpan NIM yang sama berkali-kali.

Kuis Cepat

Kelas `Mahasiswa` sudah punya urutan alami berdasarkan NIM. Dosen meminta daftar mahasiswa diurutkan dari IPK tertinggi, **tanpa** mengubah urutan alami itu. Cara yang paling tepat adalah …

A Mengubah `__lt__`/`compareTo` agar membandingkan IPK

B Memberikan `key=` (Python) atau `Comparator` (Java) berisi aturan IPK saat mengurutkan

C Membuat kelas baru `MahasiswaIPK` yang menyalin semua atribut

D Mengurutkan dengan urutan alami, lalu membalik hasilnya

Kuis Cepat: Jawaban

Kelas `Mahasiswa` sudah punya urutan alami berdasarkan NIM. Dosen meminta daftar mahasiswa diurutkan dari IPK tertinggi, **tanpa** mengubah urutan alami itu. Cara yang paling tepat adalah ...

A. Mengubah `__lt__`/`compareTo` agar membandingkan IPK

B. Memberikan `key=` (Python) atau `Comparator` (Java) berisi aturan IPK saat mengurutkan

C. Membuat kelas baru `MahasiswaIPK` yang menyalin semua atribut

D. Mengurutkan dengan urutan alami, lalu membalik hasilnya

Jawaban: B

Urutan khusus diberikan dari luar kelas, dan jumlahnya boleh sebanyak yang dibutuhkan, misalnya `sorted(daftar, key=lambda m: m.ipk, reverse=True)` atau `Comparator.comparing(Mahasiswa::getIpk).reversed()`. Pilihan A merusak urutan alami yang dipakai bagian program lain. Pilihan C berlebihan. Pilihan D hanya membalik urutan NIM.

Ringkasan

- **Koleksi** adalah objek yang menyimpan dan mengelola sekumpulan objek lain.
- ***List*** berurutan dan boleh kembar, ***tuple***/`List.of` tetap, **set** membuang kembar, **map/dict** berkunci unik.
- Di Java, `List`, `Set`, dan `Map` adalah *interface*; tulis tipe kontrak di kiri dan wujud di kanan.
- Urutan isi `set`, `HashSet`, dan `HashMap` **tidak dijamin**; pakai `dict`, `LinkedHashMap`, `TreeMap`, atau urutkan dulu.
- **Urutan alami** ditulis di dalam kelas (`__lt__`/`Comparable`); **urutan khusus** diberikan dari luar (`key=`/`Comparator`).
- `__eq__`/`equals()` **wajib** berpasangan dengan `__hash__`/`hashCode()` yang dihitung dari data yang sama.
- ***Generics*** memberi label tipe pada wadah. Java memeriksanya saat kompilasi, Python hanya menjadikannya petunjuk.

Langkah Berikutnya

- Jalankan contoh di jalur pilihanmu (Python 01—08, Java 01—09) dan studi kasus KedaiKita.
- Jalankan `hari.py` atau `Hari.java` tiga kali di folder `PBO-latihan-saya/pertemuan-12`, lalu amati urutannya.
- Kerjakan latihan, lalu cek pemahamanmu dengan kuis 10 soal.

Buka bahan pertemuan ini di GitHub:

- **Konsep**
- **Praktik Python**
- **Praktik Java**
- **Jembatan Python ↔ Java**
- **Latihan**
- **Kuis 10 soal**

github.com/antonprafanto/PBO2026/tree/master/Pertemuan-12-Koleksi-dan-Generics

Pindai untuk membuka folder pertemuan ini di GitHub